

Guía del alumno

Módulo 1, Importar y unir datos

Análisis Avanzado de Datos con Python

Cómo usar esta guía

Esta guía es para leer después de la clase, no durante. Durante la clase conviene mirar la pantalla y ejecutar, que es donde se aprende. Todo lo que se explicó en voz alta está escrito acá, con el mismo código y las mismas cifras que vieron en el laboratorio, así que no hace falta tomar apuntes de código.

Cada bloque tiene la misma estructura. Primero qué se vio y para qué sirve. Después el código celda por celda con su explicación. Al final una tabla de referencia rápida, los errores más comunes con el mensaje tal como aparece en pantalla, y tres preguntas para comprobar que quedó claro.

La última página es una hoja de referencia con todo el módulo resumido. Está pensada para imprimir y tenerla al lado en las clases siguientes.

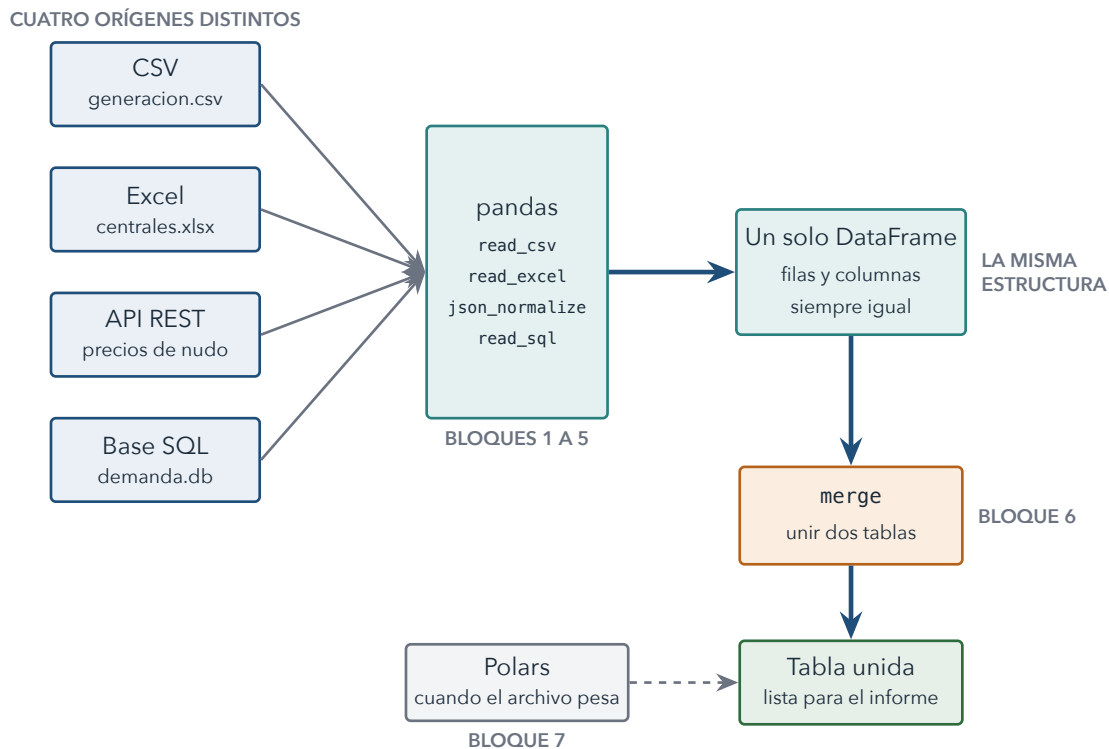
Lo que vas a poder hacer al terminar

- Cargar tablas desde un CSV, un Excel, una API y una base de datos SQL.
- Mirar una tabla nueva y saber qué preguntarle antes de usarla.
- Filtrar, agrupar y resumir con pandas.
- Unir dos tablas eligiendo el tipo de unión que corresponde, y revisar que no se haya perdido ni duplicado nada.
- Reconocer cuando conviene cambiar de herramienta porque el archivo pesa demasiado.

Todas las cifras de esta guía salen de la corrida verificada del notebook con pandas 2.3.3. El mismo notebook se ejecutó completo en Google Colab el 19 de septiembre de 2026, que hoy trae Python 3.13.15, pandas 2.2.3 y Polars 1.35.2, y las salidas coinciden. Los datos son sintéticos, hechos para el curso. Ninguna central corresponde a una instalación real.

1. El módulo en una imagen

Todo el módulo responde una pregunta. Cómo se trae a Python un dato que vive en cuatro lugares distintos, y cómo se juntan dos de esos datos sin romper las cifras.



La imagen del puerto

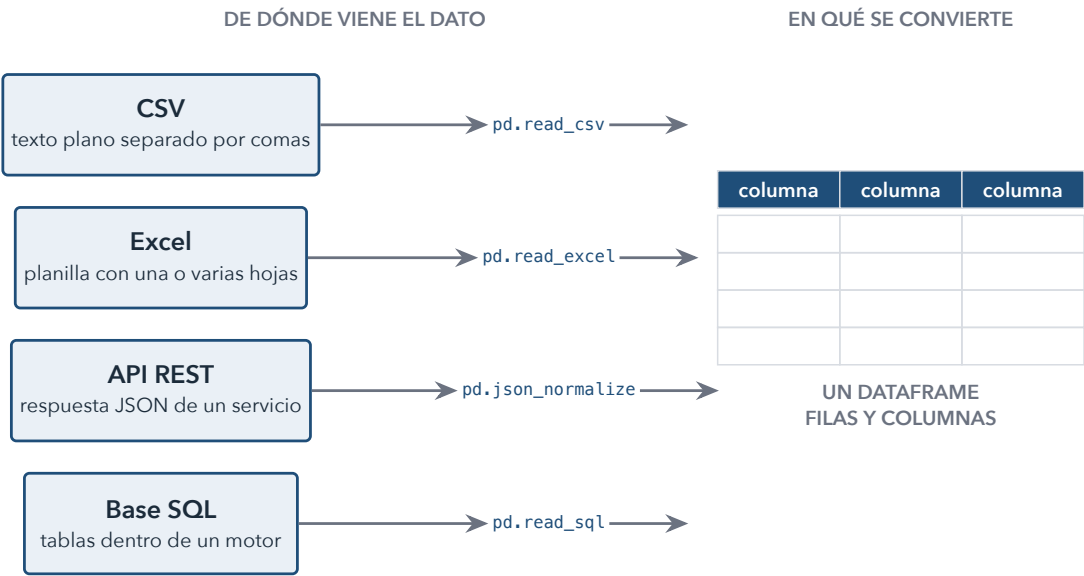
Todo este módulo pasa en un puerto.

Al puerto llega carga de todas partes y nunca llega igual. Un barco trae un contenedor sellado, otro trae pallets con sacos, una avioneta deja un sobre con documentos y desde la bodega del vecino llega un camión. Ninguna de esas cargas se puede usar tal como viene. Antes hay que bajarla al muelle, abrirla, contarla y dejarla toda en el mismo tipo de pallet, porque solo así la grúa, el camión y el inventario funcionan igual para todo.

Ese muelle es pandas. El pallet único donde queda todo es el DataFrame, la tabla de filas y columnas. El contenedor es el archivo CSV, el sobre con documentos es la respuesta de una API, los sacos son la planilla Excel y la bodega del vecino es la base de datos.

Y después viene la parte donde se gana o se pierde el informe. Para despachar hay que casar cada carga con su documentación. Hay carga que llegó sin papeles y papeles de carga que todavía no llega. Lo que uno decida hacer con esos casos es exactamente lo que en Python se llama el tipo de join, y esa decisión cambia las cifras que después se publican.

Las cuatro fuentes terminan en la misma tabla



2. Bloque 1, lo primero, una tabla

Qué se vio

Es el primer contenedor que se baja al muelle. Antes de mover un solo bulto, el que recibe hace tres preguntas. Cuántos bultos hay, qué dice el manifiesto y cuánto pesa todo junto. Esas tres preguntas, aplicadas a una tabla, son `shape`, `columns` y `sum`, y conviene hacerlas siempre, con cualquier archivo, antes de calcular nada.

La cuarta pregunta es la que sirve para trabajar. Cuánto pesa cada tipo de carga. En vez de pesar bulto por bulto, se juntan los del mismo tipo y se pesan juntos. Eso es `groupby`, que es la herramienta que más se usa en todo el curso.

Un `DataFrame` es la tabla con la que se trabaja en `pandas`. Tiene filas, columnas con nombre y un índice a la izquierda, que es la etiqueta de cada fila. Ese índice parte en cero porque Python cuenta las posiciones desde cero, y no es un dato de la tabla.

El código, paso a paso

1.1 Abrir el archivo

```
# 1.1
import pandas as pd

# read_csv lee un archivo separado por comas y lo convierte en una tabla.
# Esa tabla se llama DataFrame y es con lo que vas a trabajar todo el curso.
centrales = pd.read_csv("centrales.csv")

centrales
```

	central	tecnologia	region	potencia_mw \
0	Central Río Manso Alto	hidro	Biobío	420
1	Central Salto Verde	hidro	Los Lagos	310
2	Central Aguas Claras	hidro	Biobío	180
3	Central Vega Azul	hidro	Los Lagos	95
4	Central Tres Saltos	hidro	Biobío	260
5	Parque Solar Pampa Alta	solar	Antofagasta	230
6	Parque Solar Llano Seco	solar	Atacama	180
7	Parque Solar Sol Naciente	solar	Antofagasta	145
8	Parque Solar Quebrada Honda	solar	Atacama	95
9	Parque Solar Altiplano	solar	Antofagasta	310
10	Eólica Cerro Negro	eolica	Coquimbo	160
11	Eólica Punta Ventosa	eolica	Coquimbo	120
12	Eólica Loma Fría	eolica	Valparaíso	85
...	[la tabla sigue hasta la fila 19]			

La función `read_csv` lee un archivo separado por comas y devuelve un `DataFrame`. La tabla aparece en pantalla sin usar `print` porque el notebook muestra siempre el resultado de la última línea de la celda, y cuando ese resultado es un `DataFrame` lo dibuja con formato de tabla. Si la línea final fuera `print(centrales)` se vería lo mismo en texto plano y más incómodo de leer.

1.2 Las tres preguntas de siempre

```
# 1.2
# Tres preguntas que uno le hace a cualquier tabla apenas la abre.
print("Filas y columnas", centrales.shape)
print("Columnas      ", list(centrales.columns))
print("Potencia total ", centrales["potencia_mw"].sum(), "MW")
```

```
Filas y columnas (20, 5)
Columnas      ['central', 'tecnologia', 'region', 'potencia_mw', 'anio_inicio']
Potencia total 4540 MW
```

shape entrega primero las filas y después las columnas, en ese orden. columns entrega los nombres tal como venían en el archivo, que es donde se descubren los espacios de más y las mayúsculas raras. La suma de una columna se pide con .sum() sobre esa columna. Con esas tres respuestas uno ya sabe si el archivo es el que esperaba.

1.3 Filtrar

```
# 1.3
# Una columna se pide con su nombre entre corchetes.
# Los códigos de tecnología, como solar o eolica, van sin tilde a propósito,
# para que se puedan escribir sin errores.
# Un filtro es una condición dentro de los corchetes, que deja pasar solo las filas que la cumplen.
centrales[centrales["tecnologia"] == "solar"]
```

	central	tecnologia	region	potencia_mw \
5	Parque Solar Pampa Alta	solar	Antofagasta	230
6	Parque Solar Llano Seco	solar	Atacama	180
7	Parque Solar Sol Naciente	solar	Antofagasta	145
8	Parque Solar Quebrada Honda	solar	Atacama	95
9	Parque Solar Altiplano	solar	Antofagasta	310

	anio_inicio
5	2019
6	2020
7	2021
8	2022
9	2023

Adentro de los corchetes va una condición, y el resultado es una tabla más chica con las mismas columnas. Es el equivalente del autofiltro de una planilla, con la diferencia de que queda escrito y se puede volver a ejecutar. Fíjate en que las filas conservan su número original, del cinco al nueve, porque ese número es la etiqueta de la fila y no su posición.

Si necesitas combinar dos condiciones, cada una va entre paréntesis y se unen con & para y, o con | para o. Por ejemplo así.

```
centrales[(centrales["tecnologia"] == "solar") & (centrales["potencia_mw"] > 150)]
```

Los paréntesis no son opcionales.

1.4 Agrupar y resumir

```
# 1.4
# groupby junta las filas que comparten un valor y después las resume.
por_tecnologia = centrales.groupby("tecnologia")["potencia_mw"].sum().sort_values()

print(por_tecnologia)
```

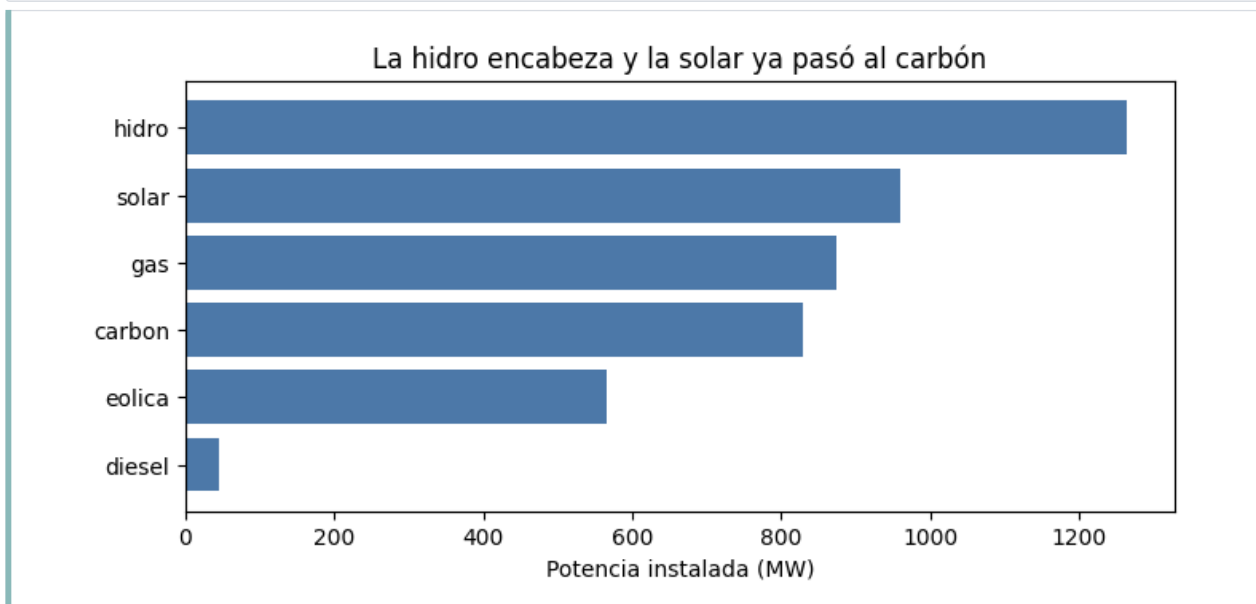
```
tecnologia
diesel      45
eolica     565
carbon     830
gas        875
solar      960
hidro     1265
Name: potencia_mw, dtype: int64
```

groupby junta las filas que comparten un valor y después aplica un resumen a cada grupo. En una planilla esto serían seis filtros y seis subtotaes. El `sort_values` de la punta ordena el resultado de menor a mayor, y para el orden inverso se escribe `sort_values(ascending=False)`.

1.5 El mismo dato, dibujado

```
# 1.5
import matplotlib.pyplot as plt

fig, ax = plt.subplots(figsize=(8, 3.5))
ax.barh(por_tecnologia.index, por_tecnologia.values, color="#4C78A8")
ax.set_title("La hidro encabeza y la solar ya pasó al carbón")
ax.set_xlabel("Potencia instalada (MW)")
plt.show()
```



El gráfico no calcula nada nuevo, dibuja el resultado que ya estaba en `por_tecnologia`. Primero el número y después el dibujo, nunca al revés. `barh` hace barras horizontales, que son las que mejor funcionan cuando las etiquetas son palabras.

OJO CON ESTA CIFRA

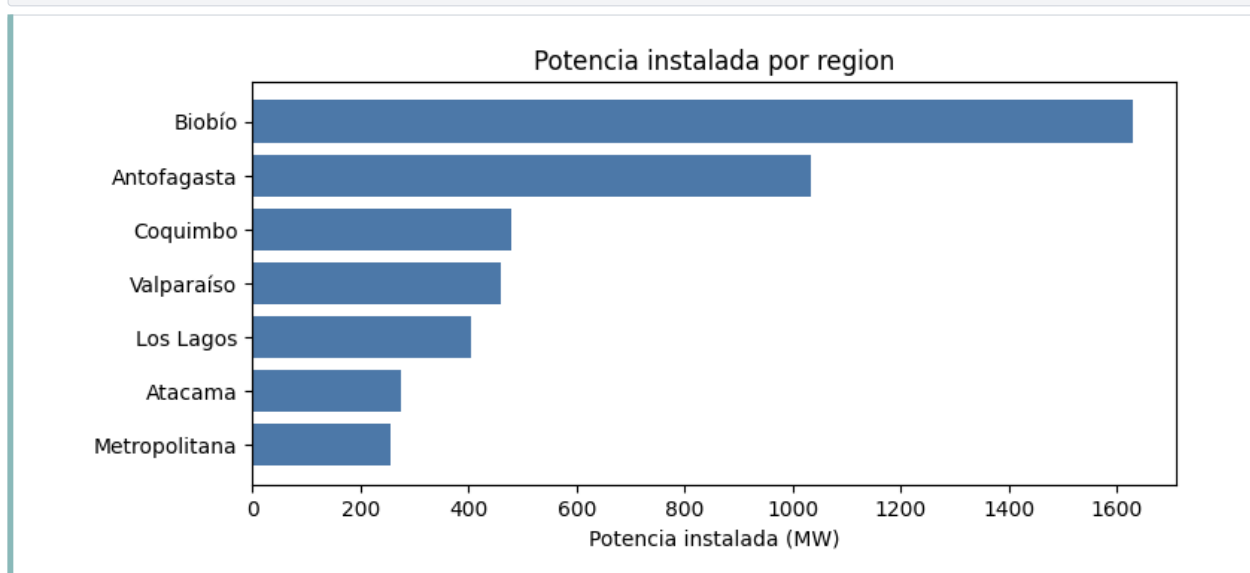
Este parque de veinte centrales es inventado para el curso y no reproduce la matriz chilena. En él la solar es segunda en potencia instalada y quinta en energía generada, porque de noche no genera. La energía por tecnología aparece en el bloque 3 del Módulo 2. Esa diferencia entre potencia y energía es justo lo que este bloque quiere mostrar. Los datos del curso son sintéticos, así que estas cifras no sirven para hablar del sistema eléctrico real.

1.8 Tu turno

```
# 1.8
columna = "region" # cámbiala por tecnologia o por anio_inicio

resumen = centrales.groupby(columna)["potencia_mw"].sum().sort_values()

fig, ax = plt.subplots(figsize=(8, 3.5))
ax.barh(resumen.index.astype(str), resumen.values, color="#4C78A8")
ax.set_title(f"Potencia instalada por {columna}")
ax.set_xlabel("Potencia instalada (MW)")
plt.show()
```



El ejercicio pedía cambiar la variable columna por tecnologia o por anio_inicio y volver a ejecutar. Al agrupar por año de inicio se ve en qué años se construyó el parque, y aparece a la vista que lo más nuevo es todo solar y eólico. El título del gráfico cambia solo porque está armado con una f-string, que reemplaza lo que va entre llaves por el valor de la variable.

Referencia rápida

Instrucción	Para qué sirve
<code>pd.read_csv(ruta)</code>	Lee un archivo separado por comas y devuelve un DataFrame
<code>df.shape</code>	Cuántas filas y cuántas columnas tiene
<code>df.columns</code>	Los nombres de las columnas
<code>df.head(n)</code>	Las primeras n filas, cinco si no se dice nada
<code>df["columna"]</code>	Una columna
<code>df[condicion]</code>	Las filas que cumplen la condición
<code>df["col"].sum()</code>	La suma de una columna, también mean, max, min
<code>df.groupby("col")["otra"].sum()</code>	Agrupar por una columna y resume otra
<code>.sort_values()</code>	Ordena, con <code>ascending=False</code> de mayor a menor

Errores comunes

Lo que aparece en pantalla	Qué pasó
<code>FileNotFoundError: [Errno 2] No such file or directory: 'centrales.csv'</code>	El nombre del archivo está mal escrito o el archivo no está donde se busca. En Colab, que no hayas corrido la celda que crea los datos.
<code>KeyError: 'potencia'</code>	Esa columna no existe con ese nombre. Revisa con <code>df.columns</code> , porque suele ser una mayúscula o un espacio de más.
<code>TypeError: can only concatenate str (not "int") to str</code>	Estás operando con una columna de texto como si fuera numérica.
<code>NameError: name 'centrales' is not defined</code>	La celda que creaba esa variable no se ejecutó, o se ejecutaron las celdas fuera de orden. Se arregla con Entorno de ejecución, Reiniciar y ejecutar todo.

Tres preguntas de repaso

1. Por qué la tabla aparece sin escribir print

Porque el notebook muestra el resultado de la última línea de la celda. Si esa línea es un DataFrame, lo dibuja como tabla.

2. Qué hace groupby y con qué se parece en una planilla

Junta las filas que comparten un valor y aplica un resumen a cada grupo. En una planilla equivale a filtrar por cada categoría y sacar un subtotal, pero en una sola línea y repetible.

3. Cómo pedirías la potencia total por región ordenada de mayor a menor

Con `centrales.groupby("region")["potencia_mw"].sum().sort_values(ascending=False)`.

3. Bloque 2, el CSV y la fecha que llega como texto

Qué se vio

El CSV es la caja con la etiqueta escrita a mano. Es un archivo de texto donde cada línea es una fila y las columnas van separadas por un carácter. Nada dentro del archivo dice cómo hay que leerlo, así que quien lo escribió decidió el separador, la codificación y el formato de las fechas, y esa decisión no viaja con el archivo.

Por eso, apenas se abre un CSV, conviene mirar los tipos de las columnas con `dtypes`. Ahí se ve el problema más silencioso de todos, que es la fecha que llegó como texto. Nada falla, la tabla está completa, y sin embargo ordenar por esa columna pone el diez de enero antes que el dos de enero, y pedir un mes es imposible sin recortar texto a mano.

LOS OTROS PROBLEMAS DE UN CSV

En este bloque aparece uno solo, la fecha que llega como texto. Casi todos los problemas al leer un CSV son uno de estos tres, los otros dos se trabajan en el Módulo 3, y los tres se resuelven en la misma línea del `read_csv`.

- `sep` cuando el archivo no usa coma sino punto y coma, que es lo habitual en Chile porque así guarda Excel en configuración regional chilena.
- `encoding` cuando los acentos llegan rotos, normalmente `encoding="latin-1"` si el archivo viene de un sistema antiguo.
- `parse_dates` cuando las fechas llegan como texto.

Hay un cuarto que aparece seguido, `decimal=","`, cuando los números vienen con coma decimal. Y un quinto para datos chilenos, `dayfirst=True`, cuando la fecha viene como día, mes y año.

El código, paso a paso

2.1 Un archivo grande

```
# 2.1
# La generación de cada central en cada hora del año.
generacion = pd.read_csv("generacion.csv")

print("Filas y columnas", generacion.shape)
generacion.head()
```

```
Filas y columnas (175680, 4)
   fecha hora      central  mwh
0  2024-01-01    0  Central Río Manso Alto  163.69
1  2024-01-01    1  Central Río Manso Alto  179.81
2  2024-01-01    2  Central Río Manso Alto  173.77
3  2024-01-01    3  Central Río Manso Alto  170.39
4  2024-01-01    4  Central Río Manso Alto  169.84
```

Son 175.680 filas, que es el resultado de veinte centrales por veinticuatro horas por trescientos sesenta y seis días, porque 2024 fue bisiesto. Con tablas de este tamaño imprimir todo no sirve de nada, y por eso se usa `head`, que muestra las primeras filas.

2.2 Mirar los tipos

```
# 2.2
# Fíjate en el tipo de cada columna. La fecha llegó como texto, no como fecha.
print(generacion.dtypes)
```

```
fecha      object
hora       int64
central    object
mwh        float64
dtype: object
```

object es como pandas escribe texto. La columna fecha llegó como texto y no como fecha, y nadie avisó nada. int64 es número entero y float64 es número con decimales, esos dos están bien.

2.3 Arreglarlo al leer

```
# 2.3
# parse_dates le dice a pandas que esa columna son fechas de verdad.
generacion = pd.read_csv("generacion.csv", parse_dates=["fecha"])

print(generacion.dtypes)
print()
print("Ahora se le pueden pedir cosas a la fecha")
print("Primer día ", generacion["fecha"].min().date())
print("Último día ", generacion["fecha"].max().date())
```

```
fecha      datetime64[ns]
hora       int64
central    object
mwh        float64
dtype: object

Ahora se le pueden pedir cosas a la fecha
Primer día    2024-01-01
Último día    2024-12-31
```

Con `parse_dates` se le indica a pandas que esa columna son fechas de verdad. El tipo pasa a `datetime64` y desde ahí se le pueden pedir cosas a la fecha, como el primer y el último día del archivo. También se puede pedir el mes con `df["fecha"].dt.month`, el día de la semana con `dt.dayofweek` o el año con `dt.year`.

Referencia rápida

Instrucción	Para qué sirve
<code>df.dtypes</code>	El tipo de cada columna
<code>sep=";"</code>	Cuando el separador no es coma
<code>encoding="latin-1"</code>	Cuando los acentos llegan rotos
<code>parse_dates=["fecha"]</code>	Cuando una columna es fecha y llega como texto
<code>decimal=","</code>	Cuando los números traen coma decimal
<code>dayfirst=True</code>	Cuando la fecha viene como día, mes y año
<code>usecols=["a", "b"]</code>	Leer solo algunas columnas, útil con archivos grandes
<code>nrows=20</code>	Leer solo las primeras filas para mirar cómo viene el archivo
<code>df["fecha"].dt.month</code>	El mes de una columna de fechas

Errores comunes

Lo que aparece en pantalla	Qué pasó
<code>ValueError: Missing column provided to 'parse_dates': 'Fecha'</code>	Le pediste convertir una columna que no existe con ese nombre exacto.
<code>UnicodeDecodeError: 'utf-8' codec can't decode byte</code>	El archivo no está en UTF-8. Prueba con <code>encoding="latin-1"</code> .
Una sola columna con todo el contenido adentro	El separador no era el que pandas supuso. Es el caso del punto y coma y se arregla con <code>sep=";"</code> . Ojo, esto no da error, simplemente queda mal.
Las fechas se ordenan mal	La columna quedó como texto. Vuelve a leer el archivo con <code>parse_dates</code> .

Tres preguntas de repaso

1. Por qué un CSV puede leerse mal sin dar ningún error

Porque el archivo no dice cómo debe leerse. Si pandas supone un separador o un tipo equivocado, igual construye una tabla, solo que no es la que corresponde.

2. Qué es lo primero que conviene mirar después de leer un archivo

El tamaño con `shape` y los tipos con `dtypes`. Las fechas y los números que aparecen como `object` son la señal de alarma.

3. Qué argumento usarías con un archivo chileno que trae punto y coma y coma decimal

Los dos juntos, `pd.read_csv(ruta, sep=";", decimal=",")`.

4. Bloque 3, Excel

Qué se vio

El Excel es el sobre con separadores adentro. Trae un archivador chico con pestañas, donde la primera suele tener la tabla y las otras traen notas, detalles o versiones anteriores. Si uno lee solo la primera hoja, se lleva la tabla y deja adentro del archivo las advertencias que explican cómo se construyó esa cifra.

Leer un Excel es la misma línea que leer un CSV con otro nombre de función. La diferencia es `sheet_name`, que elige la hoja, y que sin ese argumento pandas trae la primera.

El código, paso a paso

3.1 Leer una planilla

```
# 3.1
# read_excel funciona igual que read_csv. La diferencia es que un Excel puede traer varias hojas.
fichas = pd.read_excel("centrales.xlsx")

fichas.head(3)
```

	central	tecnologia	region	potencia_mw	anio_inicio
0	Central Río Manso Alto	hidro	Biobío	420	2004
1	Central Salto Verde	hidro	Los Lagos	310	1998
2	Central Aguas Claras	hidro	Biobío	180	2011

`read_excel` devuelve un DataFrame igual que `read_csv`. Adentro de Python, el origen del dato deja de importar.

3.2 Elegir la hoja

```
# 3.2
# sheet_name elige la hoja. Sin ese argumento pandas trae la primera.
notas = pd.read_excel("centrales.xlsx", sheet_name="notas")

notas
```

	nota
0	Potencias declaradas al 31 de diciembre de 2024
1	Las centrales de pasada se informan con su pot...

La hoja de notas de este archivo trae dos advertencias escritas por quien lo preparó. Una dice que las potencias son las declaradas al treinta y uno de diciembre de 2024, y la otra que las centrales de pasada se informan con su potencia máxima. Ese tipo de nota es lo que después explica por qué dos fuentes no dan la misma cifra.

3.3 Todas las hojas de una vez

```
# 3.3
# Con sheet_name=None trae todas las hojas de una vez, en un diccionario.
todas = pd.read_excel("centrales.xlsx", sheet_name=None)

print("Hojas del archivo", list(todas.keys()))
```

```
Hojas del archivo ['centrales', 'notas']
```

Con `sheet_name=None` llegan todas las hojas en un diccionario, donde la llave es el nombre de la hoja. Es lo primero que conviene hacer con un archivo desconocido, para saber qué hay adentro antes de elegir. Una hoja se saca con `todas["notas"]`.

Referencia rápida

Instrucción	Para qué sirve
<code>pd.read_excel(ruta)</code>	Lee la primera hoja de la planilla
<code>sheet_name="notas"</code>	Lee una hoja específica
<code>sheet_name=None</code>	Lee todas las hojas y devuelve un diccionario
<code>skiprows=3</code>	Salta las filas de título que hay antes de la tabla
<code>df.to_excel("salida.xlsx")</code>	Escribe un DataFrame a una planilla

Errores comunes

Lo que aparece en pantalla	Qué pasó
<code>ValueError: Worksheet named 'Notas' not found</code>	El nombre de la hoja no es exactamente ese. Revisa con <code>sheet_name=None</code> cuáles son.
Las primeras filas llegan como <code>Unnamed: 0</code> y datos vacíos	La tabla no empieza en la primera fila del archivo. Usa <code>skiprows</code> .
Columnas con datos faltantes donde el Excel se ve completo	Hay celdas combinadas. El valor queda en la primera celda y el resto llega vacío.

Tres preguntas de repaso**1. Qué trae pandas si no se indica la hoja**

La primera hoja del archivo.

2. Cómo se sabe qué hojas tiene una planilla desconocida

Leyéndola con `sheet_name=None` y mirando las llaves del diccionario que devuelve.

3. Pandas lee las fórmulas o los valores

Los valores guardados en el archivo. Pandas no recalcula nada.

5. Bloque 4, una API REST

Qué se vio

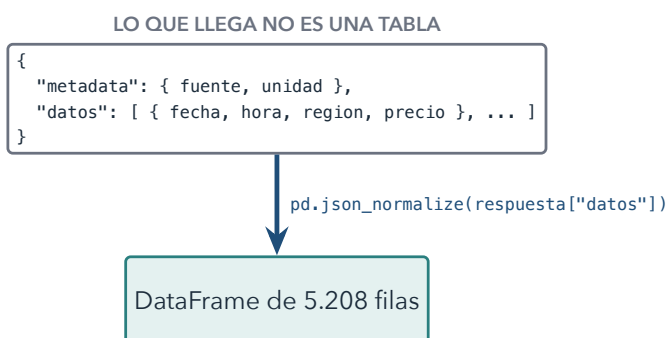
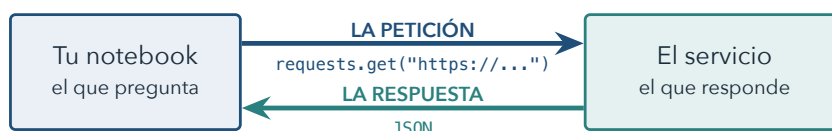
La API es la ventanilla del almacén ajeno. Hasta acá la carga llegaba sola al puerto. En este bloque uno va a buscarla. Pide con un formato acordado y recibe un sobre, también con un formato acordado. Nadie entrega una tabla lista, el trabajo de convertir ese sobre en tabla es de uno.

QUÉ ES UNA API

Una API es un mesón de atención de un sistema. Uno pide algo con un formato acordado y el sistema responde con otro formato acordado, casi siempre JSON.

La diferencia práctica con un archivo es que el dato no se baja, se pide, y el mismo código que corrió hoy corre mañana y trae el dato de mañana. Eso saca a la persona del medio del proceso, que es donde suelen atascarse los informes.

En el laboratorio la respuesta está guardada en un archivo para que funcione siempre, incluso sin internet. El código que la procesa es el mismo que usarías contra un servicio de verdad. Lo único que cambia es la primera línea, que está escrita como comentario al lado.



El código, paso a paso

4.1 Mirar cómo viene la respuesta

```
# 4.1
import json

# Contra una API real esta línea sería
# respuesta = requests.get("https://api.ejemplo.cl/precios").json()
with open("precios_nudo.json") as f:
    respuesta = json.load(f)

# Un JSON casi nunca es una tabla. Primero hay que mirar cómo viene armado.
print("Llaves de la respuesta", list(respuesta.keys()))
print(respuesta["metadata"])
```

```
Llaves de la respuesta ['metadata', 'datos']
{'fuente': 'Observatorio de Datos Energéticos', 'fecha_consulta': '2024-02-01', 'unidad': 'USD por MWh'}
```

Un JSON casi nunca es una tabla. Lo primero es mirar cómo está armado, y para eso se revisan sus llaves. Esta respuesta tiene dos, metadata con información sobre el dato y datos con la lista de registros. Contra un servicio real la primera línea sería `respuesta = requests.get(url).json()` y todo lo demás sería igual.

4.2 De sobre a tabla

```
# 4.2
# json_normalize toma la lista de registros y la aplana a tabla.
precios = pd.json_normalize(respuesta["datos"])

print("Filas y columnas", precios.shape)
precios.head()
```

```
Filas y columnas (5208, 4)
   fecha hora  region  precio_usd_mwh
0  2024-01-01  0  Antofagasta         47.97
1  2024-01-01  1  Antofagasta         40.81
2  2024-01-01  2  Antofagasta         49.35
3  2024-01-01  3  Antofagasta         45.50
4  2024-01-01  4  Antofagasta         40.00
```

`json_normalize` toma una lista de registros y la aplana a filas y columnas. Las 5.208 filas son las horas de enero por las siete regiones. De ahí en adelante es una tabla igual que cualquier otra.

Cómo se autentica una API

Casi todos los servicios de verdad piden una llave. Lo habitual es mandarla en la cabecera de la petición.

```
import requests, os

llave = os.environ["TOKEN_API"]      # la llave vive fuera del notebook
cabeceras = {"Authorization": f"Bearer {llave}"}
respuesta = requests.get("https://api.ejemplo.cl/precios", headers=cabeceras)

print(respuesta.status_code)        # 200 es correcto
datos = respuesta.json()
```

Lo importante no es la sintaxis, es dónde se guarda la llave. Nunca dentro del notebook, porque el notebook se comparte y la llave queda ahí para siempre. En Colab se usan los Secrets del panel izquierdo y en un servidor se usa una variable de entorno. Si alguna vez se publica una llave por error, hay que darla de baja, no basta con borrar la línea.

Los códigos de estado se leen por su primer dígito. Los que empiezan con dos son respuestas correctas. Los que empiezan con cuatro son un problema de la petición, por ejemplo una dirección mal escrita, una llave inválida o un permiso que falta. Los que empiezan con cinco son un problema del servidor.

Referencia rápida

Instrucción	Para qué sirve
<code>requests.get(url)</code>	Pide datos a un servicio web
<code>respuesta.status_code</code>	El código de estado, 200 es correcto
<code>respuesta.json()</code>	Convierte la respuesta en estructura de Python
<code>json.load(f)</code>	Lee un JSON desde un archivo ya abierto con <code>with</code>
<code>list(respuesta.keys())</code>	Las llaves de la respuesta, para ver cómo viene armada
<code>pd.json_normalize(lista)</code>	Convierte una lista de registros en DataFrame

Errores comunes

Lo que aparece en pantalla	Qué pasó
<code>KeyError: 'datos'</code>	La respuesta no tiene esa llave. Revisa primero con <code>list(respuesta.keys())</code> , porque cada servicio arma la respuesta a su manera.
Un DataFrame de una sola fila con columnas raras	Le pasaste a <code>json_normalize</code> la respuesta completa en vez de la lista de registros.
<code>status_code</code> igual a 401 o 403	Falta la llave de acceso o no tiene permiso para ese recurso.
<code>status_code</code> igual a 429	Hiciste demasiadas consultas en poco tiempo. Hay que esperar y espaciar las peticiones.

Tres preguntas de repaso

1.Cuál es la diferencia entre bajar un archivo y consultar una API

El archivo es una foto que alguien dejó en alguna parte. La API entrega el dato vigente al momento de preguntar, y por eso el mismo código sirve mañana sin que nadie intervenga.

2. Por qué no se escribe la llave de acceso dentro del notebook

Porque el notebook se comparte y queda registrada ahí. Se guarda en los Secrets de Colab o en una variable de entorno.

3. Qué hace `json_normalize`

Toma una lista de registros de un JSON y la convierte en un DataFrame con filas y columnas.

6. Bloque 5, una base de datos

Qué se vio

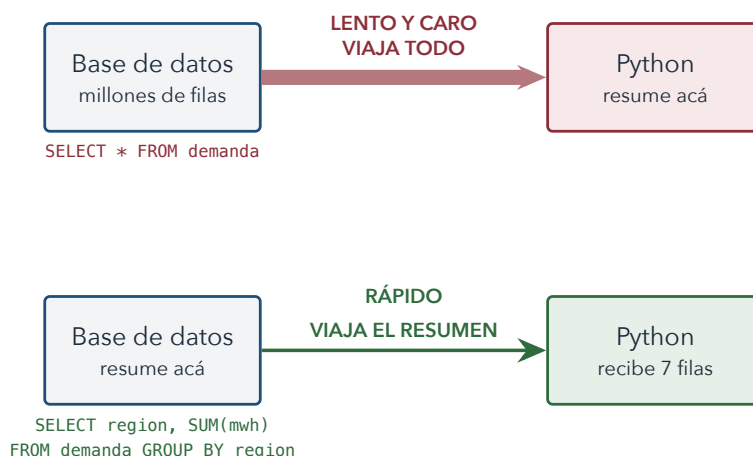
La base de datos es la bodega del vecino, la que tiene inventariador propio. Si necesitas saber cuántos sacos hay por tipo, puedes pedir que te manden todos los sacos y contarlos tú, o pedirle al inventariador que los cuente y te mande un papel con siete números. Las dos formas dan la misma respuesta. Una mueve camiones durante dos días y la otra llega en un sobre.

Conectarse a una base es una línea y consultar es otra. SQLite guarda toda la base en un archivo, mientras Postgres, SQL Server u Oracle se conectan por red, pero la línea de pandas que trae el resultado es la misma.

DÓNDE SE HACE LA CUENTA

El GROUP BY de SQL hace exactamente lo mismo que el groupby de pandas. La diferencia es dónde se hace la cuenta y cuánto dato viaja.

Si la tabla tiene cien millones de filas y tú quieres siete números, conviene que la cuenta la haga la base y que a Python lleguen siete filas. Traerse todo con un SELECT * para después resumirlo en pandas es el error más común al empezar, y es el que hace que un proceso funcione en la prueba y se caiga en producción.



El código, paso a paso

5.1 Conectarse y consultar

```
# 5.1
import sqlite3

# SQLite guarda toda la base en un archivo. Postgres u Oracle se conectan por red,
# pero la línea de pandas que viene después es exactamente la misma.
conexion = sqlite3.connect("demanda.db")

pd.read_sql("SELECT * FROM regiones", conexion)
```

	region	poblacion
0	Antofagasta	700000
1	Atacama	320000
2	Coquimbo	850000
3	Valparaíso	1900000
4	Metropolitana	8100000
5	Biobío	1700000
6	Los Lagos	900000

`sqlite3.connect` abre la base, que en este caso es un archivo. La consulta va escrita como texto y `pd.read_sql` devuelve el resultado como `DataFrame`. Con otros motores lo que cambia es la forma de armar la conexión, normalmente con `SQLAlchemy`, y no la línea de pandas.

5.2 Que la cuenta la haga la base

```
# 5.2
# El filtro se escribe en SQL y lo resuelve la base, así viaja mucho menos dato.
consulta = """
    SELECT region, SUM(mwh) AS demanda_total
    FROM demanda
    GROUP BY region
    ORDER BY demanda_total DESC
    """
pd.read_sql(consulta, conexion)
```

	region	demanda_total
0	Metropolitana	7779657.53
1	Valparaíso	1825770.20
2	Biobío	1632753.33
3	Los Lagos	864564.87
4	Coquimbo	816624.13
5	Antofagasta	672440.52
6	Atacama	307486.00

La tabla de demanda tiene 61.488 filas y esta consulta devolvió siete. El `GROUP BY` agrupa por `region`, `SUM` suma y `ORDER BY` ordena, todo dentro de la base. La Metropolitana concentra la mayor parte de la demanda, que es lo esperable por población.

Referencia rápida

Instrucción	Para qué sirve
<code>sqlite3.connect(archivo)</code>	Abre una base SQLite
<code>pd.read_sql(consulta, conexion)</code>	Ejecuta la consulta y devuelve un DataFrame
<code>SELECT columnas FROM tabla</code>	Elige qué columnas traer
<code>WHERE condicion</code>	Filtra filas dentro de la base
<code>GROUP BY columna</code>	Agrupar, equivale al groupby de pandas
<code>ORDER BY columna DESC</code>	Ordena de mayor a menor
<code>SELECT COUNT(*) FROM tabla</code>	Cuenta cuántas filas hay antes de traerlas
<code>params=(valor,)</code>	Pasa valores a la consulta sin pegarlos en el texto

Errores comunes

Lo que aparece en pantalla	Qué pasó
<code>DatabaseError: Execution failed on sql ... no such table</code>	El nombre de la tabla está mal escrito o la base es otra.
El notebook se queda pegado o se cae sin mensaje	La consulta trae demasiadas filas. Mide antes con <code>SELECT COUNT(*)</code> y filtra en la consulta.
La consulta funciona y el resultado está vacío	El filtro no calza con los datos, por ejemplo una fecha escrita en otro formato.

Tres preguntas de repaso

1. Por qué conviene resumir en la base y no en Python

Porque así viaja solo el resultado. Traerse millones de filas para producir siete números consume red, memoria y tiempo, y termina cayéndose cuando los datos crecen.

2. Qué hace `pd.read_sql`

Ejecuta una consulta SQL contra una conexión y entrega el resultado como DataFrame.

3. Dónde se guarda la contraseña de una base

Fuera del notebook, en los Secrets de Colab o en una variable de entorno del servidor.

7. Bloque 6, unir dos tablas

Qué se vio

Este es el bloque que puede equivocar una cifra publicada, y por eso es el más importante del módulo.

QUÉ ES UN JOIN

Unir dos tablas es pegarlas por una columna que ambas comparten. En el laboratorio esa columna es el nombre de la central.

La pregunta de fondo siempre es la misma, qué hago con los que aparecen en una tabla y no en la otra. La respuesta a esa pregunta es el tipo de join, y se escribe en el argumento how.

En el puerto es la aduana. La carga ya está en pallets iguales y al lado hay una carpeta con la documentación. Casi todo calza, pero siempre hay pallets sin papel y papeles de carga que no llegó. Hay cuatro decisiones posibles frente a esos casos, y son exactamente los cuatro tipos de join.

generación del día
Río Manso Alto
Cerro Negro
Loma Fría

fichas de centrales
Río Manso Alto
Cerro Negro
Proyecto Futuro

Verde, está en las dos tablas

Rojo, está en una sola

Loma Fría genera y no tiene ficha

Proyecto Futuro tiene ficha y no genera

how="inner"

Se queda solo con lo que está en las dos

Río Manso Alto
Cerro Negro

Pierde Loma Fría y Proyecto Futuro, y nadie avisa

how="left"

Conserva toda la generación

Río Manso Alto
Cerro Negro
Loma Fría, ficha vacía

Deja fuera Proyecto Futuro

how="outer"

Conserva todo de los dos lados

Río Manso Alto
Cerro Negro
Loma Fría, ficha vacía
Futuro, sin generación

No pierde nada y deja a la vista lo que falta en cada lado

how="cross"

Cruza cada fila con cada fila

Río Manso con Río Manso
Río Manso con Cerro Negro
Río Manso con Futuro

y así con las demás

3 por 3 igual a 9 filas, la grilla completa

El código, paso a paso

6.1 El escenario

```
# 6.1
# Para que se note la diferencia, trabajamos con un día y con una ficha incompleta.
un_dia = generacion[generacion["fecha"] == "2024-06-15"]

# Sacamos tres centrales de la lista de fichas y agregamos una que no genera.
fichas_parciales = centrales[~centrales["central"].isin(
    ["Eólica Loma Fría", "Central Vega Azul", "Diésel Respaldo Cordillera"])]
fichas_parciales = pd.concat([fichas_parciales, pd.DataFrame(
    [{"central": "Central Proyecto Futuro", "tecnologia": "hidro", "region": "Biobío",
     "potencia_mw": 150, "anio_inicio": 2026}]), ignore_index=True)

print("Generación de ese día", un_dia.shape[0], "filas,", un_dia["central"].nunique(), "centrales")
print("Fichas disponibles   ", fichas_parciales.shape[0], "centrales")
```

```
Generación de ese día 480 filas, 20 centrales
Fichas disponibles    18 centrales
```

Se trabaja con un solo día para que los números se puedan seguir. La generación de ese día tiene 480 filas, que son veinte centrales por veinticuatro horas. Las fichas disponibles son dieciocho, porque se sacaron tres centrales a propósito, y además se agregó una central que tiene ficha y todavía no genera. Ese es el desorden habitual entre dos fuentes que se actualizan a distinto ritmo.

6.2 inner

```
# 6.2
# inner se queda solo con las centrales que están en las dos tablas.
inner = pd.merge(un_dia, fichas_parciales, on="central", how="inner")

print("inner", inner.shape[0], "filas de", un_dia.shape[0])
print("Perdió en silencio", un_dia.shape[0] - inner.shape[0], "filas de generación")
```

```
inner 408 filas de 480
Perdió en silencio 72 filas de generación
```

inner se queda solo con las centrales que están en las dos tablas. Se perdieron 72 filas, que son tres centrales por veinticuatro horas, un quince por ciento de la generación del día. No hubo ningún error ni ninguna advertencia. Este es además el comportamiento por defecto de merge si uno no escribe how.

6.3 left

```
# 6.3
# left conserva toda la generación y deja vacía la ficha cuando no existe.
left = pd.merge(un_dia, fichas_parciales, on="central", how="left")

print("left", left.shape[0], "filas")
print("Filas sin ficha", left["tecnologia"].isna().sum())
left[left["tecnologia"].isna()]["central"].unique()
```

```
left 480 filas
Filas sin ficha 72
array(['Central Vega Azul', 'Eólica Loma Fría',
      'Diésel Respaldo Cordillera'], dtype=object)
```

left conserva todas las filas de la tabla de la izquierda, que acá es la generación, y deja vacías las columnas de la ficha cuando no la encuentra. Esos NaN son útiles, porque dicen exactamente dónde falta información. Las tres centrales sin ficha son Central Vega Azul, Eólica Loma Fría y Diésel Respaldo Cordillera.

6.4 outer

```
# 6.4
# outer conserva todo de ambos lados. Con indicator queda anotado de dónde salió cada fila.
outer = pd.merge(un_dia, fichas_parciales, on="central", how="outer", indicator=True)

print(outer["_merge"].value_counts())
```

```
_merge
both          408
left_only      72
right_only      1
Name: count, dtype: int64
```

outer conserva todo de los dos lados. Con indicator=True pandas agrega la columna _merge, que dice de dónde salió cada fila. Acá se ven las 408 filas que calzaron, las 72 que solo tienen generación y la fila que solo tiene ficha, que es la central Proyecto Futuro. Es la mejor herramienta para comparar dos catálogos.

6.5 cross

```
# 6.5
# cross cruza todo con todo. Sirve para armar la grilla completa y ver qué falta.
horas = pd.DataFrame({"hora_del_dia": range(24)})
grilla = pd.merge(fichas_parciales[["central"]], horas, how="cross")

print("Grilla completa", grilla.shape[0], "filas =",
      fichas_parciales.shape[0], "centrales por 24 horas")
```

Grilla completa 432 filas = 18 centrales por 24 horas

cross cruza cada fila de una tabla con cada fila de la otra, sin columna en común. Dieciocho centrales por veinticuatro horas dan 432 filas. Sirve para armar la grilla completa de combinaciones posibles y después ver cuáles no tienen medición.

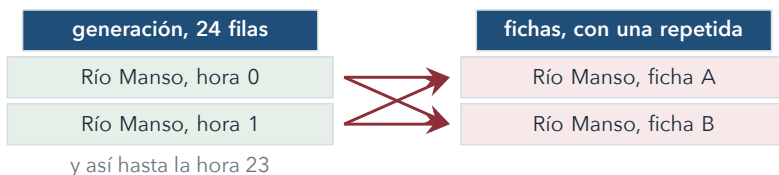
6.6 La clave repetida

```
# 6.6
# Así se revisa antes de unir.
print("Fichas repetidas", fichas_parciales["central"].duplicated().sum())

# Y así se ve el daño cuando las hay.
con_repetida = pd.concat([fichas_parciales, fichas_parciales.head(1)], ignore_index=True)
inflado = pd.merge(un_dia, con_repetida, on="central", how="inner")
print("Join normal ", inner.shape[0], "filas")
print("Join con una ficha repetida", inflado.shape[0], "filas")
```

Fichas repetidas 0
Join normal 408 filas
Join con una ficha repetida 432 filas

Esta es la trampa que conviene memorizar. Si la tabla de fichas trae una central repetida, cada fila de generación de esa central encuentra dos fichas y queda duplicada. El join pasa de 408 a 432 filas, y cualquier suma posterior queda inflada. No hay mensaje de error.



cada fila encuentra dos fichas

resultado del merge, 48 filas	
hora 0 con la ficha A	
hora 0 con la ficha B	
hora 1 con la ficha A	
hora 1 con la ficha B	

En el lab, con dos fichas repetidas, el join pasa de 480 a 528 filas y la generación del día sube de 52.650,0 a 64.298,7 MWh. Nada falla y ningún mensaje aparece. El informe simplemente queda inflado.

La revisión que lo evita, antes de cualquier merge

```
fichas["central"].duplicated().sum()
```

6.8 Tu turno, elegir el join

```
# 6.8
how = "inner"    # Pruébalo con inner, left y outer

unido = pd.merge(un_dia, fichas_parciales, on="central", how=how)

print("Generación real de ese día", round(un_dia["mwh"].sum(), 1), "MWh")
print(f"Generación con how={how:7}", round(unido["mwh"].sum(), 1), "MWh")
print()
print(unido.groupby("tecnologia")["mwh"].sum().round(1).sort_values(ascending=False))
```

```
Generación real de ese día 52650.0 MWh
Generación con how=inner    50395.9 MWh
```

```
tecnologia
hidro      18782.7
gas        13999.4
carbon     12746.9
eolica     3476.7
solar      1390.1
Name: mwh, dtype: float64
```

El ejercicio pedía cambiar `how` hasta que el total del join coincidiera con la generación real del día, que son 52.650,0 MWh. Con `inner` el total da 50.395,9 MWh, o sea faltan 2.254,1 MWh. La respuesta es `left`, que conserva toda la generación. Fíjate además en un detalle, en el resumen por tecnología las filas sin ficha no aparecen en ninguna categoría, así que la suma del resumen tampoco da el total del día. Eso es correcto y es justamente lo que hay que saber antes de publicar la cifra.

6.10 Tu turno, limpiar antes de unir

```
# 6.10
# Una tabla de fichas como llega en la práctica, con dos centrales repetidas.
fichas_sucias = pd.concat([fichas_parciales, fichas_parciales.head(2)], ignore_index=True)
print("Fichas repetidas en la tabla sucia", fichas_sucias["central"].duplicated().sum())

# Cambia esta línea por fichas_sucias.drop_duplicates(subset="central")
fichas_limpias = fichas_sucias

unido = pd.merge(un_dia, fichas_limpias, on="central", how="left")
print("Filas de generación del día", un_dia.shape[0])
print("Filas después del join      ", unido.shape[0])
print("Generación real ", round(un_dia["mwh"].sum(), 1), "MWh")
print("Generación unida", round(unido["mwh"].sum(), 1), "MWh")
```

```
Fichas repetidas en la tabla sucia 2
Filas de generación del día 480
Filas después del join      528
Generación real  52650.0 MWh
Generación unida 64298.7 MWh
```

La tabla `fichas_sucias` trae dos centrales repetidas. Tal como está, el join devuelve 528 filas donde deberían ser 480, y la generación sube de 52.650,0 a 64.298,7 MWh. La solución es dejar una sola ficha por central antes de unir.

```
fichas_limpias = fichas_sucias.drop_duplicates(subset="central")
```

Con esa línea el join vuelve a 480 filas y la generación vuelve a 52.650,0 MWh. Las dos comparaciones que imprime la celda son la verificación que conviene dejar escrita en cualquier proceso que una tablas.

La regla de los joins, en corto

- **inner** se queda con lo que está en las dos tablas y pierde el resto en silencio. Es lo que pandas hace por defecto.
- **left** conserva todas las filas de la tabla de la izquierda. Es el que corresponde cuando la tabla de la izquierda es la medición y no se puede perder nada.
- **outer** conserva todo de los dos lados y, con `indicator`, dice de dónde vino cada fila. Es el de diagnóstico.
- **cross** cruza todo con todo. Sirve para armar grillas completas, no para unir datos.
- Antes de unir se cuentan los duplicados de la clave. Después de unir se cuentan las filas. Dos líneas que evitan una cifra mal publicada.

Referencia rápida

Instrucción	Para qué sirve
<code>pd.merge(a, b, on="col", how="left")</code>	Une dos tablas por una columna común
<code>how="inner"</code>	Solo lo que está en las dos, es el valor por defecto
<code>how="left"</code>	Conserva todas las filas de la tabla izquierda
<code>how="outer"</code>	Conserva todo de los dos lados
<code>how="cross"</code>	Cruza cada fila con cada fila
<code>indicator=True</code>	Agrega la columna <code>_merge</code> con el origen de cada fila
<code>on=["fecha", "hora"]</code>	Une por más de una columna
<code>left_on, right_on</code>	Cuando la columna se llama distinto en cada tabla
<code>suffixes=("_a", "_b")</code>	Controla el nombre de las columnas repetidas
<code>validate="many_to_one"</code>	Da error si la clave no es única en la tabla derecha
<code>df["col"].duplicated().sum()</code>	Cuenta duplicados antes de unir
<code>df.drop_duplicates(subset="col")</code>	Deja una sola fila por valor de la clave
<code>df["col"].isna().sum()</code>	Cuenta las filas que quedaron sin pareja

Errores comunes

Lo que aparece en pantalla	Qué pasó
<code>MergeError: No common columns to perform merge on</code>	Las dos tablas no comparten ninguna columna con el mismo nombre. Usa <code>left_on</code> y <code>right_on</code> .
<code>MergeError: Merge keys are not unique in right dataset; not a many-to-one merge</code>	Lo dio <code>validate</code> y es una buena noticia. La clave está repetida en la tabla derecha, que es justo lo que infla el resultado.
Aparecen columnas <code>mwh_x</code> y <code>mwh_y</code>	Las dos tablas traían una columna con el mismo nombre. Contrólalo con <code>suffixes</code> o elimina la que no necesitas antes de unir.
El join devuelve más filas de las que tenía la tabla original	Hay claves repetidas. Cuenta duplicados y limpia antes de unir.
El join devuelve muchas menos filas y sin error	Es un <code>inner</code> con claves que no calzan, muchas veces por espacios de más o mayúsculas distintas en los nombres.

Tres preguntas de repaso

1. Para un informe de generación mensual, qué join corresponde y por qué

left desde la tabla de medición, porque no se puede perder generación aunque falte la ficha de una central. Las filas sin ficha quedan a la vista con NaN.

2. Cómo se detecta antes de unir que el join va a inflar el resultado

Contando duplicados en la columna de unión con `duplicated().sum()`, o dejando que pandas lo revise con `validate="many_to_one"`.

3. Si después de un inner el total baja, dónde está el problema

En las filas que no encontraron pareja. Con `how="outer"` y `indicator=True` se puede ver exactamente cuáles son y decidir qué hacer con ellas.

8. Bloque 7, cuando la tabla no cabe

Qué se vio

Es la grúa nueva del puerto. El puerto no cambia, lo que cambia es el tamaño del barco.

CUÁNDO USAR CADA UNO

Polars es más rápido porque usa todos los núcleos del computador y guarda los datos por columna. Con tablas chicas la diferencia no se nota y no vale la pena cambiar de herramienta.

La regla práctica es quedarse en pandas, que es lo que todo el mundo usa y lo que tiene más ejemplos y más librerías alrededor, y pasarse a Polars cuando el archivo pese cientos de megabytes y pandas empiece a demorar de verdad.

El resto del curso sigue en pandas.

El archivo del ejercicio tiene 4.380.000 filas y pesa 141,1 MB. Lo que se rompe primero con un archivo así no es el análisis, es abrirlo. La lectura demora y la memoria se llena.

El código, paso a paso

7.1 Crear el archivo grande

```
# 7.1
# Un archivo grande de verdad, para medir. Son cinco años de 100 centrales.
import numpy as np, os
if not os.path.exists("generacion_grande.csv"):
    rng = np.random.default_rng(7)
    n = 100 * 5 * 365 * 24
    pd.DataFrame({
        "fecha": np.repeat(pd.date_range("2020-01-01", periods=5*365).astype(str), 100*24),
        "hora": np.tile(np.repeat(np.arange(24), 100), 5*365),
        "central": np.tile([f"Central {i:03d}" for i in range(100)], 5*365*24),
        "mwh": rng.uniform(0, 400, n).round(2),
    }).to_csv("generacion_grande.csv", index=False)

print("Tamaño del archivo", round(os.path.getsize("generacion_grande.csv")/1e6, 1), "MB")
```

Tamaño del archivo 141.1 MB

La celda solo crea el archivo si no existe todavía, por eso se puede volver a ejecutar sin esperar de nuevo.

7.2 y 7.4 La misma lectura en las dos herramientas

```
%%time
# 7.2
# pandas leyendo el archivo grande
grande_pd = pd.read_csv("generacion_grande.csv")
print(grande_pd.shape)
```

```
(4380000, 4)
CPU times: user 740 ms, sys: 63.9 ms, total: 803 ms
Wall time: 807 ms
```

```
%%time
# 7.4
# Polars leyendo exactamente el mismo archivo
grande_pl = pl.read_csv("generacion_grande.csv")
print(grande_pl.shape)
```

```
(4380000, 4)
CPU times: user 292 ms, sys: 106 ms, total: 398 ms
Wall time: 29.5 ms
```

La magia `%%time` al comienzo de la celda mide cuánto demoró. El tiempo que interesa es el de pared, la última línea. Los tiempos cambian mucho entre máquinas y entre corridas, así que lo que importa es la diferencia y no el número exacto.

En Colab, Polars leyó el archivo en menos de la mitad del tiempo que pandas. Tus tiempos van a ser otros, dependen de la máquina que te toque. Lo que se mantiene es la proporción. La ventaja de Polars crece con el tamaño del archivo y con la cantidad de núcleos disponibles, y la máquina de Colab es compartida.

7.5 y 7.6 El mismo resumen en las dos

```
%%time
# 7.5
# El mismo resumen en pandas
print(grande_pd.groupby("central")["mwh"].sum().head(3))
```

```
central
Central 000    8759088.80
Central 001    8754013.08
Central 002    8746462.26
Name: mwh, dtype: float64
CPU times: user 142 ms, sys: 4.61 ms, total: 147 ms
Wall time: 147 ms
```

```
%%time
# 7.6
# El mismo resumen en Polars. Cambia cómo se escribe, no lo que hace.
# Polars no ordena el resultado si no se lo pides, pandas sí. Por eso el sort.
# 8.7591e6 es el mismo 8759088.80 de pandas, escrito en notación científica.
print(grande_pl.group_by("central").agg(pl.col("mwh").sum()).sort("central").head(3))
```

```
shape: (3, 2)
+-----+-----+
| central | mwh |
| --- | --- |
| str | f64 |
+-----+-----+
| Central 000 | 8.7591e6 |
| Central 001 | 8.7540e6 |
| Central 002 | 8.7465e6 |
+-----+-----+
CPU times: user 54.6 ms, sys: 66.6 ms, total: 121 ms
Wall time: 15 ms
```

La sintaxis cambia, `group_by` con guion bajo y `agg` en vez de la forma de pandas, y el resultado es el mismo. Polars lo imprime en notación científica, 8.7591e6 es el mismo 8.759.088,80 de pandas. En el resumen la ventaja de Polars es bastante menor que en la lectura. Fíjate en el `sort` del final. Polars no garantiza el orden del resultado de una agrupación si uno no se lo pide, y sin esa línea mostraría otras tres centrales en cada corrida. pandas, en cambio, ordena por la clave sin que se lo pidas. Por eso nunca conviene asumir el orden de una tabla.

Referencia rápida

Instrucción	Para qué sirve
<code>%time</code>	Mide cuánto demora la celda, va en la primera línea
<code>pl.read_csv(ruta)</code>	Lee un CSV con Polars
<code>df.groupby("col").agg(...)</code>	Agrupar y resume en Polars
<code>df.to_pandas()</code>	Convierte un DataFrame de Polars a pandas
<code>pl.from_pandas(df)</code>	Convierte en el sentido contrario
<code>df.to_parquet(ruta)</code>	Guarda en un formato mucho más liviano que CSV
<code>chunks=100000</code>	Lee un CSV por pedazos cuando no cabe en memoria

Tres preguntas de repaso

1. Cuándo conviene cambiarse a Polars

Cuando el archivo pesa cientos de megabytes y pandas empieza a demorar de verdad. Antes de eso, no.

2. Qué se puede hacer antes de cambiar de herramienta

Leer solo las columnas necesarias con `usecols`, filtrar el periodo en la consulta, dejar que la base entregue el resumen, leer por pedazos con `chunks` o guardar en Parquet.

3. Por qué los tiempos cambian en cada corrida

Porque dependen de la máquina, de su carga en ese momento y de si el archivo quedó en memoria caché. Lo que se mantiene es el orden de magnitud de la diferencia.

Hoja de referencia del Módulo 1

Para imprimir y tener al lado

TRAER EL DATO

<code>pd.read_csv(ruta)</code>	CSV, acepta sep, encoding, parse_dates, decimal, usecols, nrows
<code>pd.read_excel(ruta, sheet_name="hoja")</code>	Excel, con sheet_name=None trae todas las hojas
<code>pd.json_normalize(registros)</code>	JSON de una API a tabla, donde registros es la lista que trae la respuesta
<code>pd.read_sql(consulta, conexion)</code>	Base de datos, la consulta va en SQL
<code>pl.read_csv(ruta)</code>	Polars, para archivos de cientos de megabytes

MIRAR LA TABLA, SIEMPRE ANTES DE CALCULAR

<code>df.shape</code>	Filas y columnas
<code>df.columns</code>	Nombres de las columnas
<code>df.dtypes</code>	Tipos, object donde debería haber fecha o número es alarma
<code>df.head()</code>	Las primeras filas

TRABAJAR LA TABLA

<code>df[df["col"] > 100]</code>	Filtrar, cada condición entre paréntesis
<code>df.groupby("col")["otra"].sum()</code>	Agrupar y resumir
<code>.sort_values(ascending=False)</code>	Ordenar de mayor a menor
<code>df["fecha"].dt.month</code>	Pedirle el mes a una columna de fechas

UNIR DOS TABLAS, LA PARTE QUE PUEDE EQUIVOCAR UNA CIFRA

<code>df["clave"].duplicated().sum()</code>	Antes de unir, tiene que dar cero
<code>pd.merge(a, b, on="clave", how="left")</code>	Conserva todas las filas de la izquierda
<code>how="inner"</code>	Solo lo que está en las dos, pierde el resto en silencio
<code>how="outer" con indicator=True</code>	Muestra de dónde salió cada fila
<code>validate="many_to_one"</code>	Avisa con un error si la clave está repetida
<code>len(unido) == len(a)</code>	Después de unir, la comprobación de que nada se infló

Las cinco cosas que hay que recordar del módulo

- Cuatro orígenes distintos terminan siempre en la misma tabla.
- A toda tabla nueva se le preguntan el tamaño, las columnas y los tipos.
- Un CSV no dice cómo leerlo, y la fecha que llega como texto no da ningún error.
- El resumen conviene hacerlo donde están los datos, en la base y no después.
- Al unir, inner pierde filas y una clave repetida las duplica. Se cuenta antes y se cuenta después.